Rischi e checklist

1. Rischi principali

Rischio	Come si riconosce	Prevenzione	Rimedio
Contaminazione con i test	guadagni forti su una sola fonte (es. WiCE, RAGTruth)	decontaminazione in `make_mix.py` estesa a tutti i benchmark della suite; RAGTruth escluso (98% di documenti in comune)	togliere la fonte, riportare i risultati con e senza
Catastrophic forgetting	benchmark di oblio in calo; domande 2/3 peggiori; KL di replay in crescita	replay 10–15% con self-distillation (02c), lr basso	λ più alto, più replay, β·KL sul fact checking
Sovra-confidenza	ECE in aumento, soglia ottimale verso 0 o 1	loss sulle lettere (non SFT classico), 1 epoca	Brier, temperatura rifittata sul dev
Classificatore "a una stringa"	crollo con domande parafrasate o campi rinominati	varianti di domanda e campi (02d)	più varianti, quota più alta
Scorciatoie dei dati	P(supportato) bassa anche senza documento per i claim con "not"; BAcc alta con il solo claim	test "solo claim"; fonti diverse	filtrare le negazioni in FEVER (D7); più negativi per cancellazione
Formato sbagliato	togliere una fonte migliora (D1)	solo prosa (FactKG solo dopo la conversione)	ridurre o convertire la fonte
Etichette generate sbagliate	campione letto a mano con errori; il giudice LLM rifiuta molto	l'etichetta deriva dall'operazione; verifica con un secondo LLM	Brier; rivedere i prompt; scartare il task
Prompt diverso dal benchmark	la loss non scende o la BAcc peggiora subito	rendering con le funzioni dei benchmark e test token per token	correggere il rendering
Tokenizer di Winnow convertito	avviso "incorrect regex pattern" al caricamento	caricarlo come nel benchmark, test su testi con numeri e punteggiatura	confronto con il server ufficiale (`/v1/winnow/inspect`)
Documenti troncati	evidenza tagliata in righe supportate	budget 4k–8k token, conteggio dei troncati	scartare le righe o alzare il budget
Overfitting del dev	ottimo su `dev_select`, peggio su `dev_confirm`	criteri di adozione fissati prima ($\geq$ 0,5 punti)	conferma su `dev_confirm`, meno varianti provate
Fughe fra split	dev o test con documenti visti in training; dati generati da righe di test	split congelati prima di tutto (Step 0); generazione solo dal train; `doc_id` disgiunti	rifare gli split e rigenerare
AggreFact usato per decidere	scelte fatte guardando il suo dev o test	protocollo dello Step 0	dichiararlo: il risultato non è più zero-shot
Test guardato troppo presto	—	test solo dopo `frozen.json`	dichiararlo nel report
Licenze	—	vedi 08, §7	ricetta con soli dati permissivi
Conversione GGUF	probabilità diverse fra server e vLLM	controllo dei 63 esempi	rifare la conversione con la versione giusta del convertitore

2. Checklist per step

Step 0 — Split e protocollo

- ☐ Split train / dev_select / dev_confirm / test_id per ogni fonte di training, con le regole di 00b (test ufficiali dove etichettati, altrimenti per `doc_id`).
- ☐ Fonti di test_ood preparate e mai presenti in train o dev.
- ☐ Nessun `doc_id` condiviso fra split; quasi-duplicati controllati anche fra i nostri split.
- ☐ Decontaminazione contro LLM-AggreFact, test_ood e benchmark di oblio.
- ☐ `manifest.json` con hash scritto; split congelati prima di generare dati o fare la baseline.

Step 1 — Baseline

- ☐ Inferenza su `dev_select` e `dev_confirm`; soglia e temperatura fittate solo su `dev_select`.
- ☐ B2 (temperatura) e B3 (una domanda) calcolati.
- ☐ Baseline su `dev_select`/`dev_confirm`, test_id, test_ood, AggreFact zero-shot (dalle cache), test comportamentali, oblio.
- ☐ Configurazione di partenza congelata per ciascun modello.

Step 2a — Fonti esistenti

- ☐ `build_pairs.py all` eseguito; statistiche per fonte salvate.
- ☐ Decontaminazione contro tutti i benchmark della suite.
- ☐ 20 righe per fonte lette a mano.

Step 2b — Generazione con LLM

- ☐ Generazione avviata solo da righe dello split di train (Step 0).
- ☐ Modello generatore e modello giudice scelti (diversi, se possibile).
- ☐ Tasso di accettazione misurato su ≥ 200 esempi per task.
- ☐ 30 esempi accettati letti a mano: < 3 errori.
- ☐ Coppie verificate salvate come nuove fonti, con `doc_id` ereditato.

Step 2c — Replay

- ☐ Domande da `typed-decisions` (solo train) e da altre fonti, nello schema `criteria`.
- ☐ Nessuna domanda di fact checking nel replay.
- ☐ Target calcolati con il modello di partenza congelato, separatamente per Winnow e JPT.
- ☐ Split di validazione del replay tenuto da parte.

Step 2d — Varianti di domanda

- ☐ ~60 varianti generate per domanda.
- ☐ Filtro di significato con il modello di partenza.
- ☐ ~10 varianti per domanda tenute da parte per la valutazione.

Step 2e — Mix

- ☐ Ricetta salvata (JSON) e mix prodotto.
- ☐ Positivi 45–55%; nessun `doc_id` in comune con dev e test dello Step 0; troncati $< 1\%$.

Step 3 — Rendering

- ☐ Prompt identici a quelli dei benchmark (test su 20 righe di 5 fonti).
- ☐ Id delle lettere verificati (Winnow 236776/236799; JPT 357/417).
- ☐ Target nel giusto ordine (Winnow A = falso; JPT A = sì).

Step 4 — Loss

- ☐ Funzioni testate sui casi numerici di 04_loss.md.

Step 5 — Training

- ☐ Configurazione e seed salvati.
- ☐ Monitoraggio a ogni checkpoint: BAcc, ECE, KL di replay, "solo claim".
- ☐ Checkpoint scelto con il criterio di 05, §8.

Step 6 — Valutazione

- ☐ Soglia rifittata su `dev_select`, confermata su `dev_confirm`; soglia secondaria di AggreFact dichiarata.
- ☐ `dev_select`, test comportamentali e oblio a ogni esperimento; test_id, test_ood e AggreFact solo per la baseline e alla fine.
- ☐ Intervalli di confidenza per documento.

Step 7 — Esperimenti

- ☐ Ogni corsa registrata con ID, variazione, metriche e decisione.
- ☐ Controlli C1–C3 eseguiti.

Step 8 — Merge ed export

- ☐ Fusione su pesi bf16; verifica PEFT contro fuso superata.
- ☐ (Winnow) GGUF BF16 e Q8_0; controllo dei 63 esempi superato.
- ☐ `decision.json` e `frozen.json` scritti prima del test.
- ☐ Test eseguito una volta; report e model card pubblicati.